Final Capstone Report — Module 7.1

Student submission: Paste each section below into the corresponding field in *Self-study Capstone Checkpoint Activity 7.1: Final Capstone Report Planning Template*.

---

1. Project Title

CRE Research Agent: An Agentic RAG System for Commercial Real Estate Intelligence

---

2. Problem and User

Commercial real estate (CRE) analysts and investment professionals must answer complex, multi-source research questions—vacancy trends, cap rates, labor markets, lease economics—by manually cross-referencing hundreds of PDF market reports, brokerage publications, and structured datasets. A typical question such as *"Compare office vacancy across sunbelt markets and identify submarkets with the strongest rent growth acceleration"* requires searching multiple documents, reconciling entity names (e.g., DFW vs. Dallas-Fort Worth), extracting tables, and synthesizing findings with verifiable citations.

The intended user is a **CRE research analyst** who needs evidence-backed answers, not plausible prose. In this domain, an ungrounded vacancy figure can influence investment decisions worth millions of dollars, so reliability and provenance matter as much as fluency.

A standalone language model is insufficient because: (1) the corpus exceeds any context window, (2) market data goes stale quickly, (3) comparative questions require multi-hop retrieval and tool use, and (4) analysts require exact citations (document, page) rather than paraphrased summaries. This problem matters because it sits at the intersection of high-stakes financial judgment and information overload—the exact class of workflow where autonomous systems must retrieve, reason, and self-correct rather than guess.

---

3. System Goal and Scope

Goal: Transform multi-step CRE research into a conversational, tool-augmented workflow that plans retrieval, executes specialized analysis, synthesizes cited recommendations, and surfaces uncertainty when evidence is thin.

Successful performance looks like:

- Every factual claim linked to a retrieved source
- Complex comparative queries analyzed across cost, talent, and risk pillars
- Self-critique scores above threshold, or explicit escalation when below
- Responses within latency targets (< 30s simple, < 60s complex)

Scope boundaries:

- Domain: CRE and location intelligence only (not legal, tax, or personal financial advice)
- SQL: read-only (`SELECT`) by default; DDL requires explicit approval
- Retrieval: hybrid vector search over indexed market documents; page numbers used for citation, not as a retrieval filter
- Synthesis by default; exact table/figure pull reserved for evidence-mode requests

Constraints:

- Maximum plan length: 5 steps
- Maximum refinement iterations: 10
- Minimum sources before confident synthesis: 3
- ToT synthesis: feature-flagged; activates only when ≥3 pillar workers complete

Design rationale:

- **Agentic, not prompt-only** — CRE answers must be grounded, current, and cited; a single LLM call cannot retrieve, iterate, or self-correct, so an agent loop is required.
- **ReAct + self-critique** — explicit Plan → Execute → Observe → Refine is auditable and bounds failure modes (no silent hallucination, no infinite loops).
- **Specialized workers** — cost, talent, and risk pull on different evidence; one generalist worker dilutes each. Parallel specialists improve depth without raising latency.
- **ToT only for complex queries** — beam-3 synthesis avoids premature commitment on multi-pillar comparisons, but the latency/cost is not justified for simple lookups, so it is feature-flagged and gated by a complexity check.
- **Read-only by default, escalate by default** — high-impact actions (DDL, low-confidence answers, conflicting sources) defer to a human; safety bias is toward over-escalation, not over-autonomy.
- **Hard caps over soft heuristics** — plan length, iteration cap, and minimum-source threshold are numeric and enforced, making the system predictable in production.

---

4. Final System Architecture

The system is an integrated **agentic RAG platform** with three phases:

Phase 1 — Planning: An LLM planner decomposes the user query into typed execution steps (`retrieve`, `worker_cost`, `worker_talent`, `worker_risk`, `synthesize`, etc.) with a dependency graph. Independent steps run in parallel.

Phase 2 — Execution (ReAct): Each step follows Act → Observe → Evaluate → Refine. The agent calls tools (vector search, SQL, chart/map generation), stores results in working memory, and inserts refinement steps when quality thresholds are not met.

Phase 3 — Synthesis: Evidence from workers and retrieval is merged into a cited response. For complex multi-pillar queries, a **Tree of Thought (ToT)** layer replaces linear synthesis. ToT in four steps:

1. **Generate** three parallel branches, each through a different lens (cost, talent, risk).
2. **Score** each branch with self-critique on relevance, support, usefulness.
3. **Prune** any branch scoring below 0.5.
4. **Select** the highest-scoring branch as the final response (or show both if the top two are within 0.1 of each other).

Major components and how they integrate:

Component	Role
Planner (Driver)	Creates execution plan; gates simple vs. complex paths
Retriever	Hybrid BM25 + dense vector search; populates working memory
Cost / Talent / Risk Workers	Parallel pillar analysis; structured JSON with claims, metrics, `missing_data`
Working Memory	Short-term facts, deduplicated sources, step results within one run
Conversation History	Long-term multi-turn context for follow-up queries
Self-Critique (Critic)	Scores relevance, support, usefulness; triggers refinement
ToT Synthesizer	Beam-3 multi-lens synthesis for complex comparisons
Guardrails	Domain scope, input sanitization, citation enforcement, iteration caps
Logging	Color-coded stages (Planning, Fetch, Execution, Synthesis, Quality)

Simple queries stay on the linear ReAct path. Complex queries activate workers + optional ToT. Safety and evaluation layers wrap the entire flow—guardrails constrain behavior, metrics measure quality, and human intervention criteria trigger when confidence falls below 0.5 or sources conflict.

End-to-end example. Query: *"Compare Dallas vs. Phoenix for a 50,000 SF regional headquarters — consider cost, talent, and risk."*

1. **Plan** — Planner emits 5 steps: `retrieve`, `worker_cost`, `worker_talent`, `worker_risk`, `synthesize`.
2. **Retrieve** — Hybrid vector search returns chunks from market reports for both cities with citations.
3. **Workers (parallel)** — Cost analyzes lease rates and opex; Talent analyzes labor pool and wages; Risk analyzes volatility and climate. Each produces JSON with cited claims and a `missing_data` field.
4. **ToT synthesis** — Worker output is compacted into an evidence block. Three branches generate competing recommendations; Critic scores them; the cost branch wins (0.85) with talent runner-up (0.78, within tie threshold).

5. **Response** — Final answer shows both branches with citations: *"Primary lens: Cost — Dallas. Alternative view: Talent — Phoenix."*

---

5. Design Evolution Across the Program

Module	Checkpoint	Key refinement
1	1.1 Scoping	Framed CRE analyst problem; defined environment (vector index, SQL, tools); established plan-then-execute with feedback loops
2	2.1 Reasoning & memory	Made ReAct explicit (plan → execute → self-critique); separated working memory vs. conversation history; justified vector search + SQL as grounding/computation tools
3	3.1 RAG	Declared retrieval required; specified hybrid search, chunking (1500 chars, 200 overlap), top-k=20; mitigated low-relevance retrieval via reformulation + min-source threshold
4	4.1 ToT	Identified premature commitment in linear synthesis; designed beam-3 cost/talent/risk branches with critic pruning; gated ToT to complex queries only
5	5.1 Multi-agent	Formalized six roles (Planner, Retriever, 3 Workers, Critic); hybrid graph + iterative coordination via shared working memory
6	6.1 Safety	Defined guardrails, evaluation metrics, human escalation at confidence < 0.5; defense-in-depth strategy

Most important evolution: Moving from "RAG chatbot" (Module 3) to **structured multi-agent research** (Modules 5–6) with optional ToT for comparative synthesis. Early design treated synthesis as a single LLM call; final design recognizes that cost, talent, and risk pull on different evidence and require parallel workers plus branch exploration before committing to a recommendation.

---

6. Implementation Overview

Frameworks and platform:

- **FastAPI** backend with streaming SSE chat (production deployment)
- **Next.js** conversational UI
- **Databricks** Unity Catalog, Vector Search (hybrid), SQL Warehouse, model serving endpoints
- **LLM:** Claude via Databricks serving (`databricks-claude-sonnet-4`); endpoint is swappable through the `LLM_ENDPOINT` environment variable in `config.py`
- **Observability:** MLflow tracing (`ENABLE_TRACING=true` by default) instruments each agent stage—planning, retrieval, worker execution, synthesis—for live span inspection and offline evaluation

Core Python modules (public repo):

- `agentic_orchestrator.py` — ReAct loop, plan execution, ToT hook
- `tot_synthesizer.py` — Parallel branch generation, scoring, pruning, winner selection
- `agent_state.py` — Execution plan, working memory, step enums
- `agent_tools.py` — Vector search, SQL, chart/map, worker dispatch interface
- `config.py` — Feature flags (`ENABLE_TOT_SYNTHESIS`, `ENABLE_AGENTIC_MODE`)
- `logging_setup.py` — Stage-colored observability

How they support the design:

- Planner output maps to `ExecutionStep` objects with `StepAction` types
- Workers write structured `worker_output` JSON consumed by ToT `build_evidence_block()`
- Self-critique function plugs into both linear refinement and ToT branch scoring
- Feature flags keep ToT off by default to control latency/cost on simple queries

The public GitHub repository contains a **sanitized skeleton** of the agent core (no proprietary prompts, hosts, or internal schema names). It is sufficient for reviewers to understand architecture and validate ToT/orchestrator logic via automated tests.

---

7. Evaluation and Results

Automated tests (public repo): 12/12 passed (`pytest src/tests/ -v`)

Test suite	Validates
`test_tot_synthesizer.py` (9 tests)	Prune threshold 0.5, winner selection, tie handling, fallback when all branches fail
`test_tot_orchestrator_integration.py` (3 tests)	ToT not called when flag off; not called below 3 workers; called when enabled + threshold met

Design metrics (production targets from Checkpoint 6.1):

Metric	Target
Groundedness (% cited claims)	> 95%
Source support score	> 0.7
Relevance score	> 0.7
Latency	< 30s simple / < 60s complex
Fallback rate	< 5%
Missing data transparency	100% when evidence thin

Qualitative evaluation: Sample queries in `examples/sample_queries.md` and `examples/expected_outputs.md` document expected behavior for simple (linear ReAct), complex (workers + ToT), low-confidence fallback, and out-of-domain rejection.

Live observability: MLflow tracing captures per-stage spans (plan, retrieve, workers, synthesize, critique) so groundedness, latency, and fallback rate can be measured against the targets above when the system is connected to a live vector index.

Strengths:

- Clear plan → execute → synthesize separation
- ToT avoids premature commitment on multi-pillar comparisons
- Graceful fallback to linear synthesis when all ToT branches prune
- Feature-flagged complexity gating limits cost on simple queries

Limitations:

- Public repo does not include live vector index or full FastAPI/frontend stack
- Groundedness and latency targets are design goals; full benchmark runs require connected deployment
- Orchestrator stubs in public repo use `NotImplementedError` for tool dispatch—logic validated via mocked integration tests

---

8. Safety and Reliability Considerations

Guardrails:

- Domain scope filter redirects non-CRE queries
- Input sanitization before search and SQL
- Mandatory `[Source: file, page]` citation format; uncited claims lower support score
- SQL read-only by default; DDL gated behind approval
- Iteration cap (10) and per-step token budget (1500)
- Minimum source threshold (3) with gap warning on thin evidence

Monitoring and fallback:

- Structured logging tags each stage (Planning, Fetch, Execution, Synthesis, Quality)
- Self-critique loop scores every synthesis; triggers re-retrieval on low support
- ToT falls back to linear synthesis if all branches prune or error
- Retrieval reformulation on low relevance scores

Human oversight triggers:

- Confidence < 0.5 after refinement exhausted
- Conflicting high-stakes sources (both presented with citations)
- Out-of-scope legal/tax/financial advice
- DDL/write operations
- Zero retrieval results after reformulation retries

Defense-in-depth: guardrails constrain actions proactively; metrics measure outcomes reactively; intervention criteria define when the agent must stop and defer to an analyst.

---

9. Limitations and Next Steps

Current limitations:

1. Public repo is a sanitized subset—not the full production deployment
2. ToT in production adds 3–6 LLM calls per complex query (latency/cost trade-off)
3. Worker outputs depend on retrieval quality; low-relevance chunks still risk weak synthesis
4. Planned tools (asset library retrieval, DDL materialization) not yet in public skeleton
5. Live evaluation of groundedness/latency requires connected Databricks environment

Next steps:

1. Expand evaluation harness with golden-query set and human rubric scoring
2. Add early-convergence detection in ToT when all branches agree (reduce redundant LLM calls)
3. Wire asset library for exact-evidence mode (tables/figures by ID)
4. Publish anonymized benchmark results against design metrics
5. Harden human-in-the-loop UI for low-confidence and conflict flags

---

10. Public GitHub Repository

URL: https://github.com/rahulsingh-data-ai/cmu-capstone-cre-agent

Contents for technical reviewers:

Item	Location
README (project, architecture, setup, usage)	`README.md`
Core agent code	`src/agent/`
Automated tests	`src/tests/` (12 tests)
Architecture detail	`docs/architecture.md`
Evaluation summary	`docs/evaluation.md`
Sample queries	`examples/sample_queries.md`
Expected output shapes	`examples/expected_outputs.md`
Test results artifact	`examples/test_results.txt`

Suggested review order: Start with `README.md` for the overview, then read `src/agent/agentic_orchestrator.py` and `src/agent/tot_synthesizer.py` for the core logic, then `src/tests/` to see what is validated.

How to review locally:

```
git clone https://github.com/rahulsingh-data-ai/cmu-capstone-cre-agent.git cd
cmu-capstone-cre-agent pip install -r requirements.txt pytest src/tests/ -v python
src/agent/main_demo.py
```

No cloud credentials required for test validation or demo stub.